

MediScanX: Design of Algorithms (P3)

Sardar Patel Institute of Technology, Mumbai. Department of Computer Engineering, Mini Project, Semester V, 2026-27

Vidhaan Shah (2024300212), Gaurang Satone (2024300205), Jay Pradhan (2024300184).
Guide: Prof. Anand Godbole

1. Introduction

In the SRS we split MediScanX into ten modules. This document explains how the main processing steps in modules M3 to M8 will actually work, written as pseudocode with a short note on each about running time and why we made the choice we did. Nothing here is final code, the numbers (thresholds, epochs, filter order) are the values we plan to start with and will adjust after seeing validation results.

The algorithms are listed below and each one points back to the SRS requirement it covers.

No.	Algorithm	SRS requirement
A1	Upload validation	FR-3.1 to 3.4
A2	Chest image preprocessing	FR-4.1, 4.3
A3	ECG preprocessing	FR-4.2, 4.3
A4	Imaging model training and thresholds	FR-5.1, 5.2
A5	ECG model training	FR-6.1
A6	Heart rate from R-peaks	FR-6.3
A7	Decision-level fusion	FR-7.1 to 7.3
A8	Grad-CAM heatmaps	FR-8.1, 8.3, 8.4
A9	Full analysis pipeline	AC-1 to AC-5

In the complexity notes, n is the number of pixels in an image (224×224), T is the number of ECG samples per lead (1000) and L is the number of leads (12).

2. A1: Upload validation

Before anything touches a model we check the file, so a wrong upload gives the user a clear message instead of a crash later. The check is on content (can the file really be decoded?) and not only on the extension, because people rename files.

```
function validate_upload(file, kind):
    if file.size > 20 MB:
        return error("file too large")
    if kind == image:
        if extension not in [jpg, jpeg, png, dcm]:
            return error("unsupported image type")
        img = decode(file)          # pydicom for dcm, OpenCV for the rest
        if decode failed:
            return error("image is corrupt")
        if min(img.height, img.width) < 128:
            return error("image too small")
        if extension == dcm:
            remove patient tags from the header
    else:
        if extension == csv:
            signal = read_csv(file)  # one column per lead
        else if extension in [hea, dat]:
            signal = wfdb.rdrecord(file)
        else:
            return error("unsupported ECG type")
        if signal has 12 leads is false:
            return error("ECG must have 12 leads")
    save sha256 of the file
    return ok
```

It only reads the file once, so the time is proportional to the file size (at most 20 MB).

3. A2: Chest image preprocessing

The DenseNet we use was pretrained on ImageNet, so the X-ray has to be turned into the same kind of input: three channels, 224 x 224, normalised with the ImageNet mean and standard deviation. CLAHE is optional and helps on dull, low-contrast scans. We also catch images that are almost blank.

```

function preprocess_image(img, use_clahe):
    if img is dicom:
        convert to 8-bit grayscale
    if img has 3 channels:
        img = grayscale(img)
    if use_clahe:
        img = clahe(img, clip = 2.0, tiles = 8x8)
    img = resize(img, 224, 224)
    poor_quality = std_dev(img) < 10
    x = img / 255
    x = copy x into 3 channels
    x = (x - imagenet_mean) / imagenet_std
    return x, poor_quality

```

Every step is one pass over the pixels, so it is $O(n)$. The limit of 10 grey levels for "low contrast" is a guess for now and we will tune it by trying a few bad images.

4. A3: ECG preprocessing

PTB-XL is at 100 Hz, so whatever the user uploads is brought to 100 Hz, band-pass filtered between 0.5 and 40 Hz to remove baseline wander and high-frequency noise, normalised lead by lead and cut or padded to exactly 10 seconds (12 x 1000). We use zero-phase filtering (filtfilt) so the QRS peaks do not shift in time, which matters later for A6.

```

function preprocess_ecg(signal, fs):
    if fs != 100:
        signal = resample(signal, from = fs, to = 100)
    b, a = butterworth(order = 4, band = [0.5, 40], fs = 100)
    for each lead in signal:
        lead = filtfilt(b, a, lead)
        if std_dev(lead) is almost 0: mark lead as flat
        if too much high-frequency energy: mark lead as noisy
        lead = (lead - mean(lead)) / (std_dev(lead) + small_value)
    if length > 1000: keep first 1000 samples
    else: pad with zeros up to 1000
    return signal, lead_flags

```

Filtering a lead is linear in its length, so the whole thing is $O(L \times T)$.

5. A4: Imaging model (training and thresholds)

This is the CheXNet-style model: DenseNet-121 with ImageNet weights and the last layer swapped for 14 sigmoid outputs, since one X-ray can show several findings at once. ChestX-ray14 is very imbalanced (most labels are rare), so each label gets a positive weight equal to negatives divided by positives in the loss. We train with Adam at $1e-4$, batch size 32, and stop when validation AUROC has not improved for 3 epochs. Flips are not used for augmentation because they swap the side of the heart.

```
function train_imaging():
    model = densenet121(pretrained = imagenet)
    model.last_layer = linear(1024 -> 14) + sigmoid
    for each label k:
        pos_weight[k] = count_negative[k] / count_positive[k]
    best_auc = 0, bad_epochs = 0
    repeat for each epoch:
        for each batch (x, y) in train set:
            x = augment(x)                # small rotation, crop, brightness
            loss = weighted_bce(model(x), y, pos_weight)
            update weights with Adam
        auc = mean AUROC over 14 labels on validation set
        if auc > best_auc:
            best_auc = auc, save checkpoint, bad_epochs = 0
        else:
            bad_epochs = bad_epochs + 1
        if bad_epochs == 3: stop
    return best checkpoint

function pick_thresholds(model, validation_set):
    for each label k:
        draw the ROC curve for label k
        threshold[k] = point where (sensitivity + specificity - 1) is largest
    return threshold
```

The threshold for each label is chosen with Youden's J statistic on the validation set. Training cost is epochs x images x cost of one DenseNet pass, and it is done offline on a free Colab/Kaggle GPU. Picking thresholds needs sorting the scores for the ROC curve, so about $V \log V$ per label for V validation images. At serving time one X-ray is a single forward pass on CPU.

6. A5: ECG model (training)

A small 1D residual CNN, much lighter than the models in the PTB-XL benchmark papers, because we are limited to free GPU time. Four residual blocks, global average pooling and 5 sigmoid outputs for NORM, MI, STTC, CD and HYP. We use the official folds: 1 to 8 for training, 9 for validation, 10 for testing.

```
function ecg_net(x):                                # x is 12 x 1000
    h = relu(batchnorm(conv1d(x, 64 filters, kernel 7)))
    for block in 1..4:
        r = relu(batchnorm(conv1d(h)))
        r = batchnorm(conv1d(r))
        h = relu(h + shortcut(r))                  # stride 2 from block 2 onwards
    h = global_average_pool(h)
    return sigmoid(linear(h -> 5))

function train_ecg():
    for each epoch:
        train on folds 1-8 with binary cross entropy and Adam
        macro_auc = macro AUROC on fold 9
        save checkpoint if macro_auc improved
        stop if no improvement for 5 epochs
    report macro AUROC on fold 10
```

A forward pass is linear in the signal length. Since the later blocks use stride 2, the signal gets shorter each time and most of the work happens in the first block. The skip connections are there so training stays stable even though the network is fairly deep for such a short input.

7. A6: Heart rate

We find the R-peaks on lead II and turn the gaps between them into beats per minute (FR-6.3). The signal is first squared after a 5 to 15 Hz filter so the QRS complex stands out, then scipy's find_peaks picks the tall points, with a minimum distance of 0.25 s between two peaks so one beat is not counted twice.

```

function heart_rate(lead_II, fs = 100):
    s = square(bandpass(lead_II, 5, 15))
    peaks = find_peaks(s, height = 0.5 * max(s), distance = 0.25 * fs)
    if number of peaks < 2:
        return "not available"
    gaps = differences between consecutive peaks, in seconds
    return 60 / median(gaps)

```

The running time is $O(T)$. We take the median of the gaps and not the mean, so a single missed or extra peak does not change the answer much.

8. A7: Fusion of the two branches

There is no public dataset with an X-ray and an ECG of the same patient, so the two models are trained separately and only combined at the end (SRS section 4.4). For the image side we take the highest of Cardiomegaly, Edema and Effusion, and for the ECG side we take 1 minus the NORM probability. The two are averaged with the validation AUROC of each branch as its weight, so the stronger model counts for more.

```

function fuse(img_probs, ecg_probs, auc_img, auc_ecg):
    if image given:
        p_img = max(img_probs[Cardiomegaly], img_probs[Edema],
img_probs[Effusion])
    if ecg given:
        p_ecg = 1 - ecg_probs[NORM]
    if both given:
        S = (auc_img * p_img + auc_ecg * p_ecg) / (auc_img + auc_ecg)
        disagree = abs(p_img - p_ecg) > 0.5
    else if only image: S = p_img, disagree = false
    else if only ecg: S = p_ecg, disagree = false
    band = "Low" if S < 0.33, "Moderate" if S < 0.66, otherwise "High"
    return S, band, disagree

```

This is constant time. The band cut-offs 0.33 and 0.66 are only for how the result is displayed and the SRS does not fix them, so we simply split the 0 to 1 range into three equal parts and may change this later. The result is always shown as a screening indicator and never as a diagnosis. Feature-level fusion (joining the 1024-d image embedding with the ECG vector) stays a stretch goal if we get MIMIC access.

9. A8: Grad-CAM

To show where the model was looking we use Grad-CAM. For the chosen class we take the gradient of its score with respect to the feature maps of the last convolution layer, average each gradient map to get one weight per channel, add up the feature maps using those weights and keep only the positive part (ReLU). The result is scaled up to the image size, normalised to 0 to 1 and drawn over the X-ray as a colour map. The same steps on the last 1D convolution of the ECG model give a curve along the time axis that we draw on the lead plot.

```
function grad_cam(model, x, class_c, last_conv):
    run forward pass, keep feature maps A[1..K]
    score = output[class_c]
    run backward pass to get gradients G[1..K] of score w.r.t. A
    for each channel k:
        alpha[k] = average of G[k] over all positions
    cam = relu(sum over k of alpha[k] * A[k])
    cam = resize(cam, input size)
    cam = (cam - min(cam)) / (max(cam) - min(cam) + small_value)
    return cam

function overlay(xray, cam, opacity):
    return (1 - opacity) * xray + opacity * colormap(cam)
```

It costs one forward and one backward pass plus a weighted sum over the $K = 1024$ channels of DenseNet-121. By default it is done for the top predicted label and the user can pick another one. Every heatmap carries the line that it shows model attention and not a proven cause (FR-8.4).

10. A9: The whole analysis

This is what happens behind the analyze button (POST /cases/{id}/analyze), putting A1 to A8 together. One or both files can be missing, the steps for a missing one are skipped.

```

function analyze_case(case_id, user):
    case = load case, check that user is allowed to see it
    if case has no image and no ecg:
        return error("nothing to analyse")
    write "analysis started" to audit log
    if image: x_img, img_flag = preprocess_image(image)
    if ecg:   x_ecg, ecg_flags = preprocess_ecg(ecg)
    run these two at the same time:
        if image: img_probs = imaging_model(x_img), mark findings above
thresholds
        if ecg:   ecg_probs = ecg_model(x_ecg), hr = heart_rate(ecg)
    S, band, disagree = fuse(img_probs, ecg_probs, auc_img, auc_ecg)
    if image: heatmap = grad_cam(imaging model, x_img, top label)
    if ecg:   saliency = grad_cam(ecg model, x_ecg, top class)
    save predictions, model version, inference time, heatmap files
    write "analysis finished" to audit log
    return results
(everything is inside try/except so a failure gives a readable error and the
server keeps running)

```

The cost of one case is basically two forward passes and two backward passes (for Grad-CAM) on fixed-size inputs. The two branches do not depend on each other, so running them in parallel saves time on the 4 vCPU server.

11. How we will check them

- A4 and A5: AUROC per label and macro AUROC on the held-out test sets, plus sensitivity, specificity and F1 at the chosen thresholds. Targets from the SRS are 0.80 (imaging) and 0.87 (ECG), and we will report what we really get.
- A7: compare image only, ECG only and fusion wherever we have data for it.
- A8: look at the heatmaps on a sample of the ChestX-ray14 images that have bounding boxes. This is only a visual sanity check.
- A1 to A3 and A6: pytest unit tests with good, bad and borderline inputs.

12. Limits

- Thresholds, the low-contrast limit and the fusion bands are starting values and will change after validation.

- ChestX-ray14 labels were mined from text reports and are noisy, which is why we report AUROC and not accuracy.
- Fusion is a weighted average, not a jointly trained model, so it assumes both branches speak about the same cardiac problem.
- MediScanX is a college prototype and a second opinion aid. It is not a medical device.